

By submitting this exam I affirm I have read and complied with all instructions and requirements herein.

Oyinkansola Olayinka

885255182

Name (required)

CWID (required)

The Graph of Set C is a comparison between the operations of Searching into a BST, DLL, SLL, Hash Table and a Vector. Comparing all the movements of the graphs a have a lot of similarity to me. To start with the basic Big O complexity for this operation. Searching a BST has a time complexity of $O(\log N)$ with the worst case of $O(n)$ when the tree is unbalanced. Looking at the line plot for BST we can sort of see some similar movement that a logarithmic graph would look like with some squiggly lines. I believe the reason for the high spike would be when it has reached its worst case of being unbalanced which is linear. Searching a DLL has the time complexity of $O(1)$ with a worst case for searching a DLL is having to traverse the entire to list to find the value that is being looked for. Reason because a DLL is an unsorted list and would have to traverse the whole list. Looking at the plotted line for DLL it has a trend of starting low and then later rising up but not as much compared to the other graph. This could be because it started with a search of constant and when it got to the worse case it began to rise up higher. Searching of a SLL has the time complexity of $O(n)$ similar to DLL, the worst case for searching a SLL would be $O(1)$. But a DLL has a more advantage over the SLL with the ability to iterate back and forth, this is what makes them different from one another. Looking at line plot of SLL I notice it starts slow and has a linear trend upwards which is similar to its time complexity. The time complexity for searching a hash table on average would be $O(1)$ if it is properly implemented, but during its worse case when a collision happens it has a time complexity of $O(n)$ when there is n amount of elements in a hash table. Looking at the way the line is plotted in set C the graph starts with a straight constant trend and goes up gradually. It is a bit confusing to me why it goes up this could be related to the worst case being linear that gives it a slant trend in the graph. The time complexity for searching a vector is $O(n)$ with a worse case of having to iterate through the whole vector. Looking at the graph of the vector it has the lowest depth and as a linear rise similar to its complexity. Something Interesting I found from the graph is Searching of a vector seems to have the lowest spike, the reason for this could be because elements in a vector are contiguous in memory which could be more easier to keep track when looking for elements. I also noticed that searching a BST has the highest spike in the graph, this could be because when it becomes unbalanced the worst case. This would normally occur when elements are inserted in an ascending order that makes the graph spike really high. There is also a really close similarity between the SLL and DLL and I think the reason DLL was a got a little bit higher towards the trend would be the how it would be much more complex to search a DLL compared to a SLL. Searching a Hash Table and BST are also similar together looking at the graph but a BST is more higher, this could be because the worst case scenario for a BST $O(n)$ could be more better than the worst case scenario of a Hash table $O(n)$ because of the collision. Although they both have the same time complexity the collision that occurs within a hash table is less favorable with the case of a tree that is not balanced, which could be easier to resolve. Towards the end of the graph there is downtrend in which all of the lines dropped, I believe this could be an error on my part for including the section, but I think this could also be with the change in numbers that is associated with the downtrend, could be that the numbers in the data set got smaller. I noticed there is a squiggly line which I am unsure why they all have this trend in them, could be a fluctuation in numbers.

A real word example for this would be how we search and access contact information in the phone app, WhatsApp, or a contact documented software. Where informations could be accessed by name, date saved, alphabetically, etc. How contact is displayed in a specific order in the phone app is an example of using a vector to maintain the order in which contacts are saved. Searching for a contact information can be associated with a hash table, vector, DLL, SLL , and BST.

The best data structure that would suit this real world example would be a hash table to search for contact information as fast as possible, hash table has a time complexity of $O(1)$ on average which is more faster for users compared to SLL, BST, DLL and a vector. Making it more suitable for users when they want to access their contact information really fast.